

Report

Summary table

Parameter	Simulated results
Optimal Vdd	0.32v
Critical path delay	7.4347e-09 s
Min leakage power	3.083720e-07 W
Max leakage power	8.064468e-07 W
Avg total power	2.902443e-06 W
Avg PDP	2.902443e-14 J
Avg EDP	2.902443e-22 J*s

Critical path

The critical path with inputs $A=0xFFFFFFFF$ and $B=0x00000000$ and $Cin=1$ is the worst case scenario for my adder. This is because when A is all 1s and B is all 0s, the adder is in the propagate state, so when Cin is 1, the carry out $Cout$ value must ripple through all 32 bits of my adder, which is the longest possible path.

Wires + parasitics

My design involves using a long wire to support the carry propagate signal through all the bit slices from bit 0 to bit 31. This negatively impacts the critical path of my adder due to the parasitics a long wire introduces into the design. Since wire resistance is a function of its length, the long wire length results in a higher wire resistance, which in turn results in more power drawn, more delay ($T = RC$), and possible signal degradation due to voltage drop. Similarly, wire capacitance is also a function of its length. Our long wire in turn results in a high capacitance. Resulting in even more delay ($T=RC$). Additionally, the

long wire could also introduce capacitive coupling between adjacent wires, adding even more parasitic delay on top of the intrinsic wire delay.

Min and max leakage power

Min leakage power: 3.083720e-07 W

Max leakage power: 8.064468e-07 W

The minimum leakage power occurred when $A=0x00000000$, $B=0x00000000$, and $C_{in}=0$. In this state, all the NMOS transistors were turned off, and were able to benefit from the stacked transistor effect to prevent more current from leaking to ground. The multiple stacked transistors were able to decrease the amount of leakage current, thus the amount of leakage power, and this result is reflected in my calculations for minimum leakage power.

The maximum leakage power occurred when $A=0xFFFFFFFF$, $B=0x00000000$, and $C_{in}=1$. In this state, the worst case scenario was achieved where the only off NMOS device was the final device in the stack right before ground. This dramatically increases leakage current, as the benefit of decreased leakage current due to stacked transistors can no longer be enjoyed since only 1 device is off and it is the final device in the stack before ground.

Total average power

For calculating my total average power, my simulation methodology was to use 100 randomly generated input vectors. I felt this was sufficient because the randomness of my input vectors modeled the real world application of my adder, and I felt 100 inputs was enough to get enough test cases for a reasonable estimate of average power. I chose to not specifically use edge cases because they do not represent the consistent real world inputs of my adder.

The screenshot displays the Cadence Virtuoso schematic editor interface. The main workspace shows a detailed CMOS circuit schematic. The circuit consists of several stages of logic, including inverters and NAND gates, implemented using PMOS and NMOS transistors. Key components include:

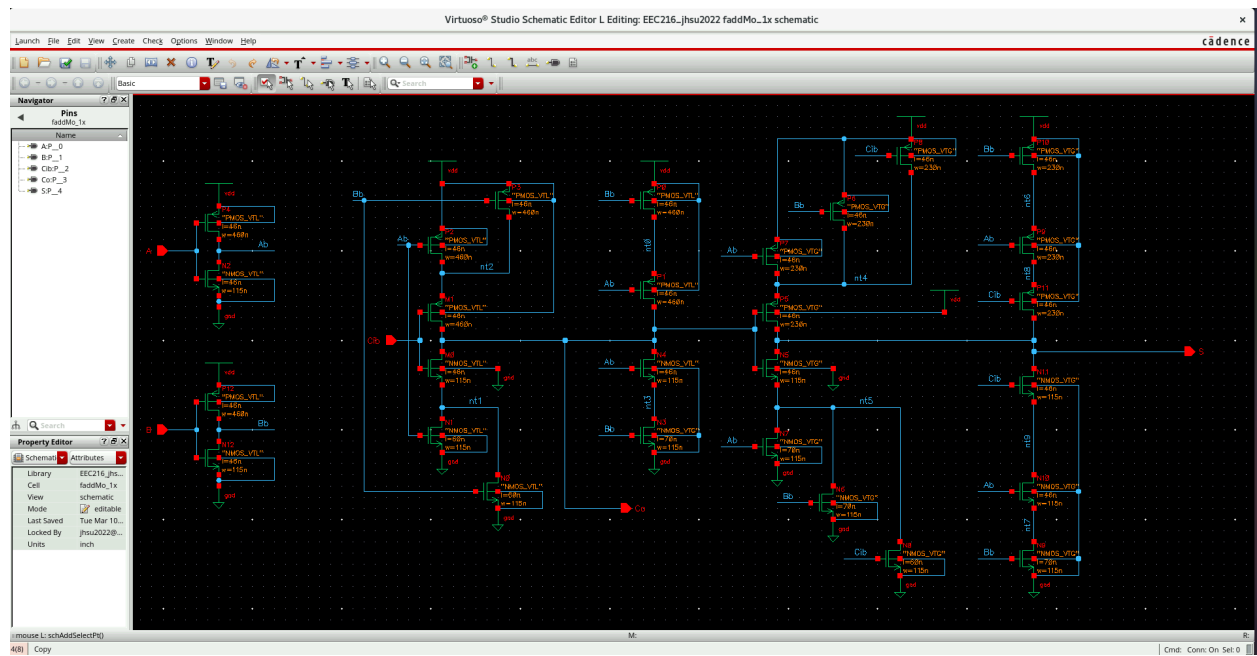
- Transistors:** Labeled with names like PMOS_VT1, NMOS_VT1, etc., and parameters such as width (w) and length (l).
- Capacitors:** Labeled with names like C1, C2, etc., and values in femtofarads (f).
- Logic Gates:** Represented by standard logic symbols for inverters and NAND gates.
- Inputs/Outputs:** The circuit has four main pins: A, B, C, and S, which are connected to various parts of the circuit.

The left sidebar shows the 'Pins' panel with the following inputs:

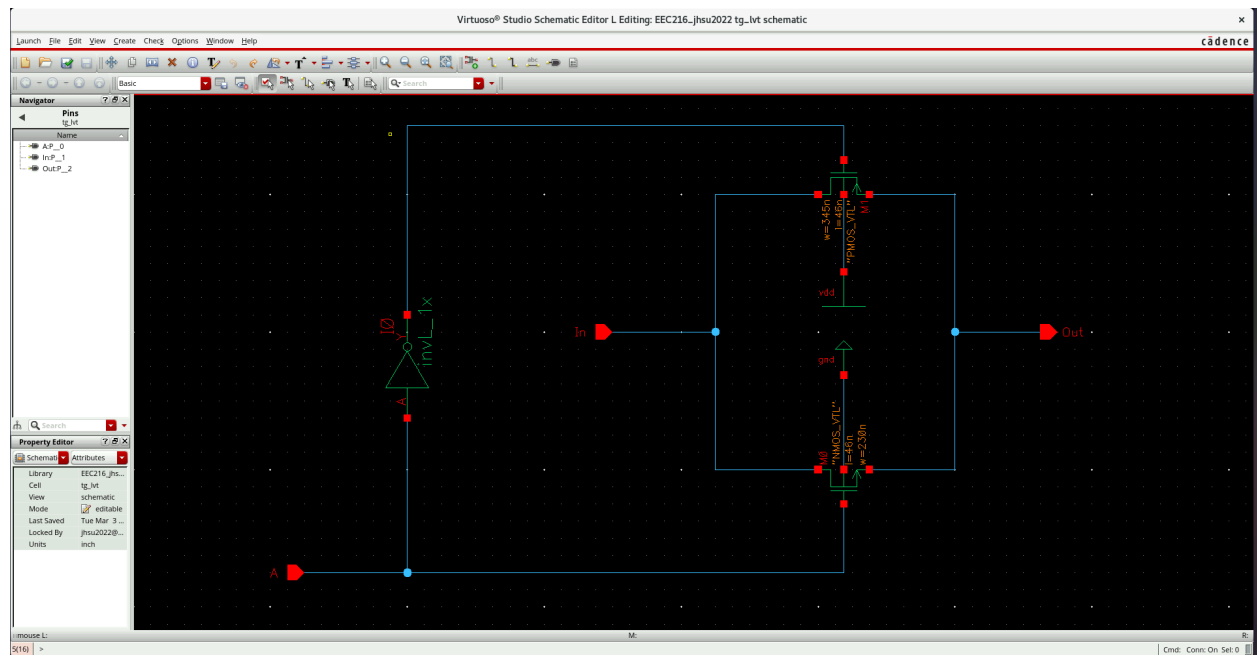
- A_P_1
- B_P_0
- C_P_2
- Cou_P_3
- S_P_4

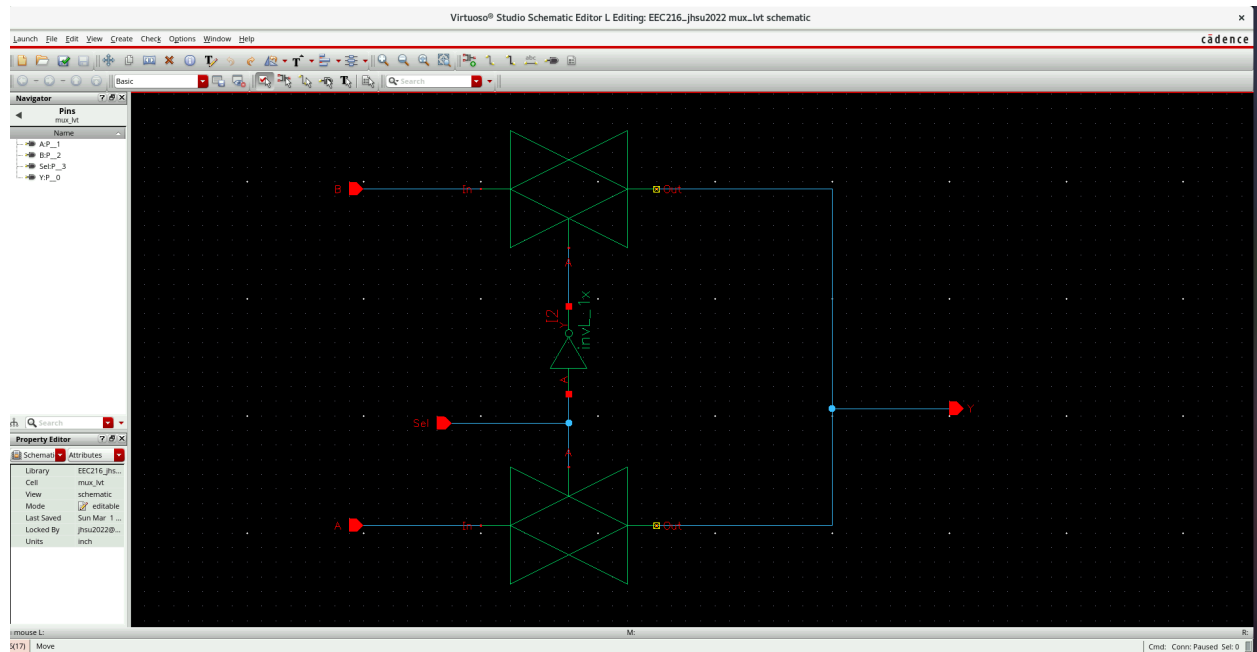
The bottom status bar indicates the mouse is over a 'Select' component.

Odd adder



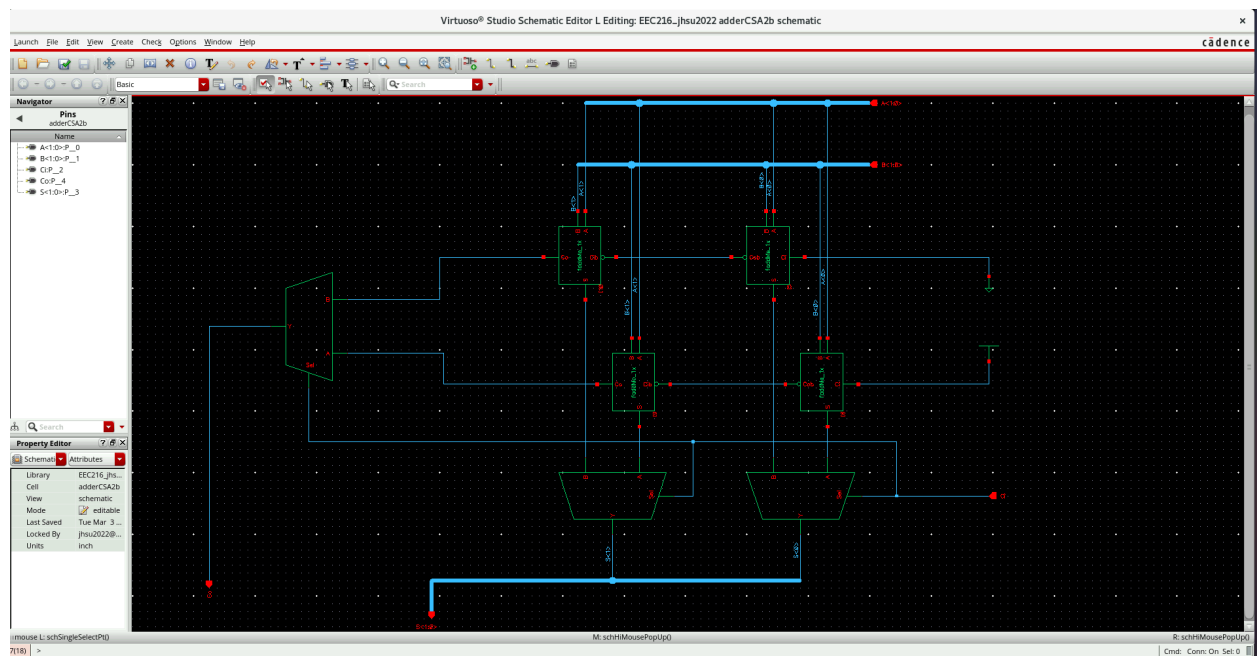
Transmission Gate + Mux



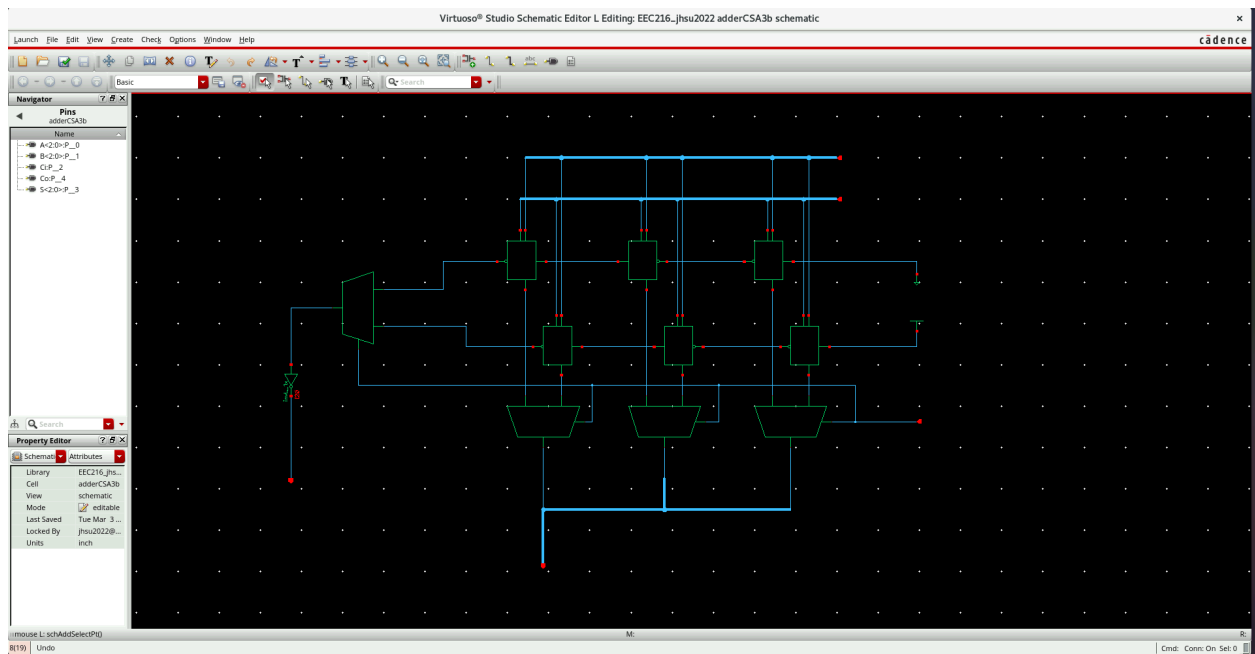


Carry select adder cells

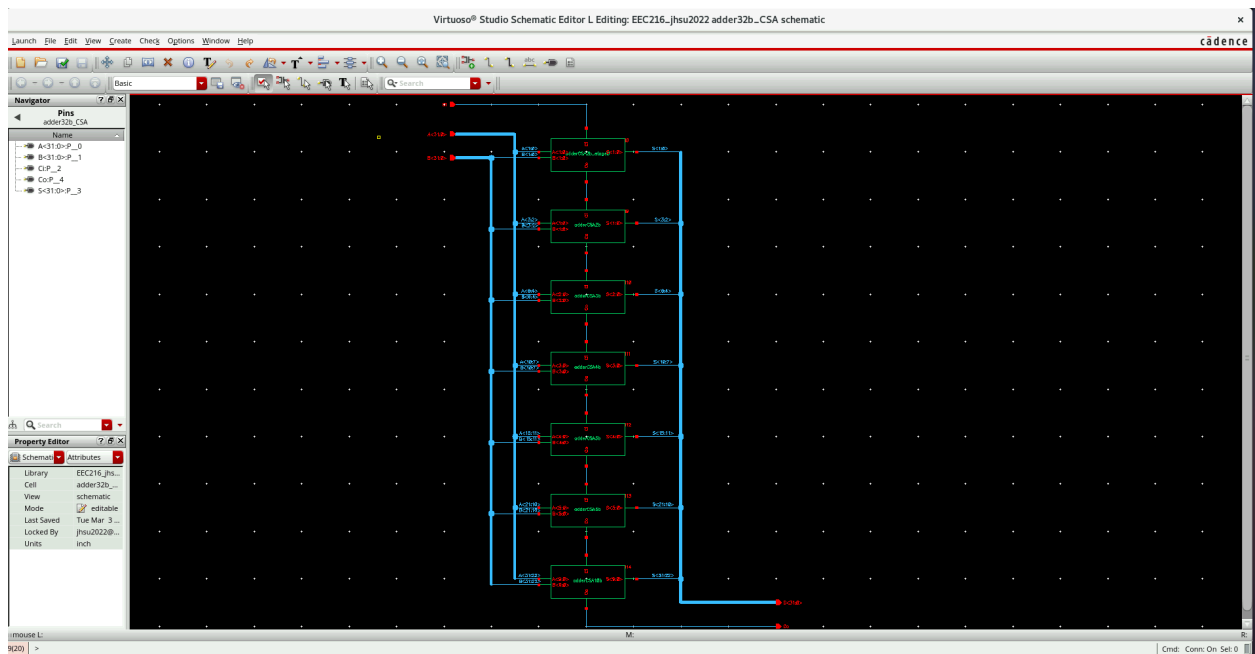
2b carry select



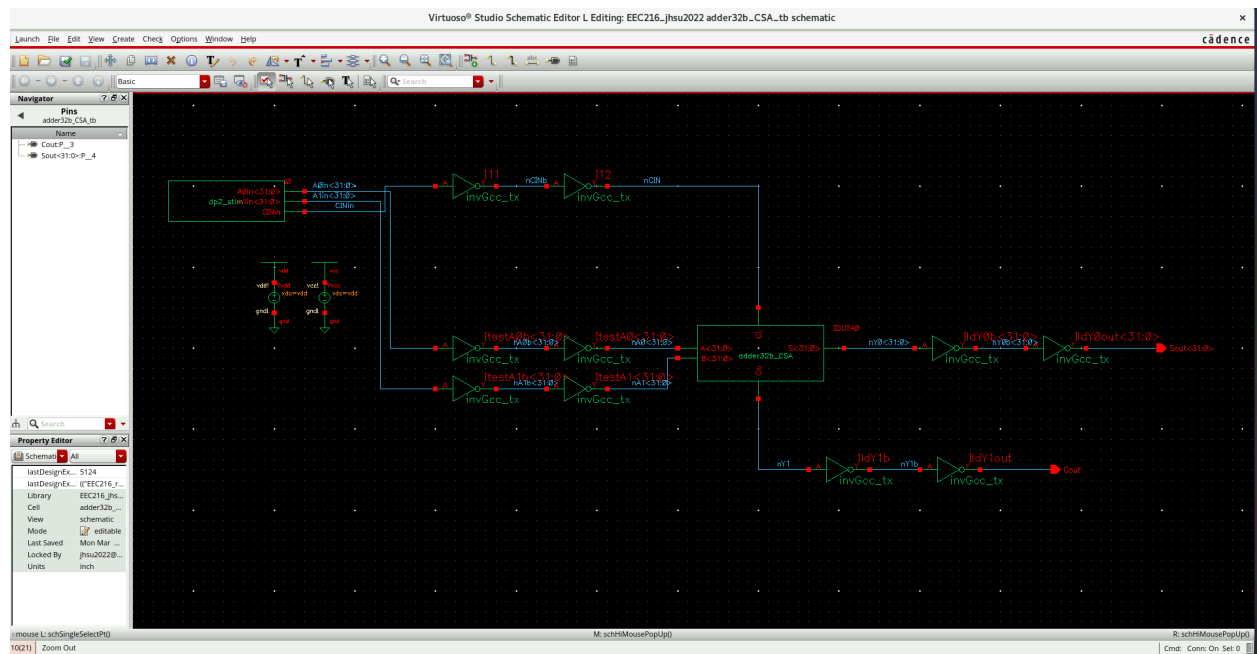
3b carry select



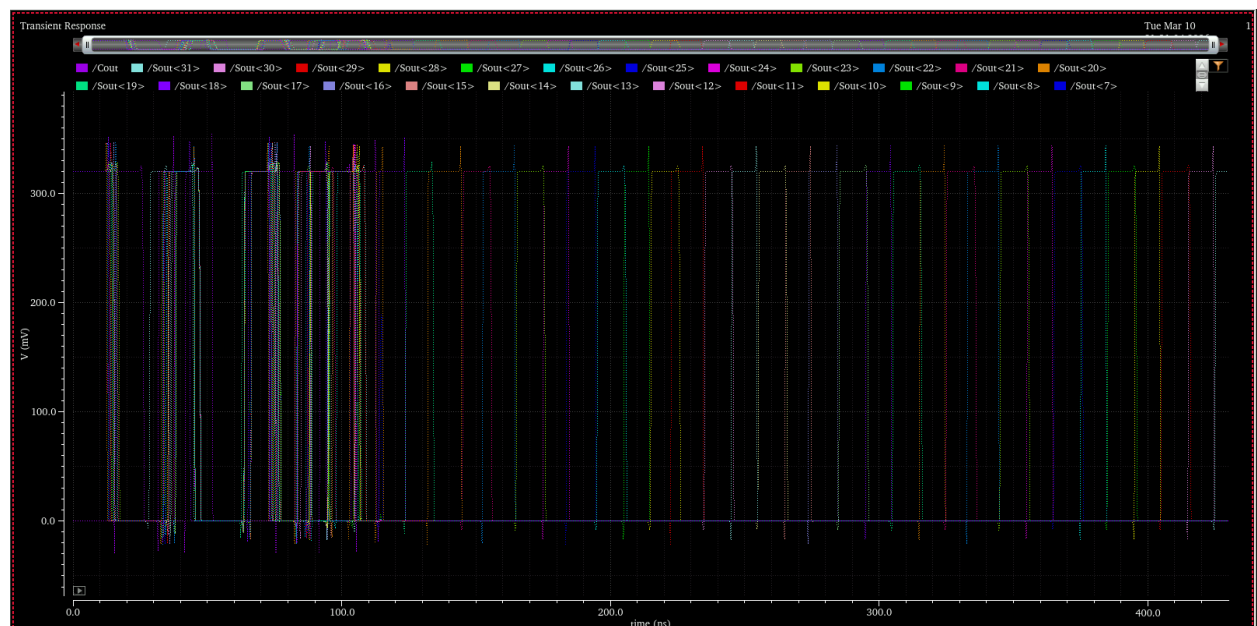
32b square root carry select adder (2-2-3-4-5-6-10)



Testbench for 32b carry select adder



Output verification



Starting 32-bit Adder Verification

--- Evaluating VDD = 0.32 V (VOH >= 0.288 V, VOL <= 0.032 V) ---

// Test 1: 0xFFFFFFFF + 0x00000000 + CIN=0 = 0xFFFFFFFF

[PASS] @ 9.9 ns

Expected -> Sout: 0xFFFFFFFF | Cout: 0

Actual -> Sout: 0xFFFFFFFF | Cout: 0

// Test 2: 0xFFFFFFFF + 0x00000000 + CIN=1 = 0x00000000, Cout=1

[PASS] @ 19.9 ns

Expected -> Sout: 0x00000000 | Cout: 1

Actual -> Sout: 0x00000000 | Cout: 1

// Test 3: 0x7FFFFFFF + 0x00000000 + CIN=1 = 0x80000000

[PASS] @ 29.9 ns

Expected -> Sout: 0x80000000 | Cout: 0

Actual -> Sout: 0x80000000 | Cout: 0

// Test 4: 0xFFFFFFFF + 0x00000000 + CIN=1 = 0xFFFFFFFF

[PASS] @ 39.9 ns

Expected -> Sout: 0xFFFFFFFF | Cout: 0

Actual -> Sout: 0xFFFFFFFF | Cout: 0

// Test 5: 0x80000000 + 0x80000000 + CIN=0 = 0x00000000, Cout=1

[PASS] @ 49.9 ns

Expected -> Sout: 0x00000000 | Cout: 1

Actual -> Sout: 0x00000000 | Cout: 1

// Test 6: 0x00000000 + 0x00000000 + CIN=0 = 0x00000000

[PASS] @ 59.9 ns

Expected -> Sout: 0x00000000 | Cout: 0

Actual -> Sout: 0x00000000 | Cout: 0

// Test 7: 0xAAAAAAAA + 0x55555555 + CIN=0 = 0xFFFFFFFF

[PASS] @ 69.9 ns

Expected -> Sout: 0xFFFFFFFF | Cout: 0

Actual -> Sout: 0xFFFFFFFF | Cout: 0

// Test 8: 0xAAAAAAAA + 0x55555555 + CIN=1 = 0x00000000, Cout=1

[PASS] @ 79.9 ns

Expected -> Sout: 0x00000000 | Cout: 1

Actual -> Sout: 0x00000000 | Cout: 1

```
// Test 9: 0x2468ACF0 + 0x13579BDF + CIN=0 = 0x37C048CF
[PASS] @ 89.9 ns
    Expected -> Sout: 0x37C048CF | Cout: 0
    Actual   -> Sout: 0x37C048CF | Cout: 0

// Test 10: 0x9ABCDEF0 + 0x9ABCDEF0 + CIN=0 = 0x3579BDE0, Cout=1
[PASS] @ 99.9 ns
    Expected -> Sout: 0x3579BDE0 | Cout: 1
    Actual   -> Sout: 0x3579BDE0 | Cout: 1

// Test 11: 0x12345678 + 0xEDCBA987 + CIN=1 = 0x00000000, Cout=1
[PASS] @ 109.9 ns
    Expected -> Sout: 0x00000000 | Cout: 1
    Actual   -> Sout: 0x00000000 | Cout: 1

// Test 12: j=0 (A0=0x00000000, CIN=1)
[PASS] @ 119.9 ns
    Expected -> Sout: 0x00000001 | Cout: 0
    Actual   -> Sout: 0x00000001 | Cout: 0

// Test 13: j=1 (A0=0x00000001, CIN=1)
[PASS] @ 129.9 ns
    Expected -> Sout: 0x00000002 | Cout: 0
    Actual   -> Sout: 0x00000002 | Cout: 0

// Test 14: j=2 (A0=0x00000003, CIN=1)
[PASS] @ 139.9 ns
    Expected -> Sout: 0x00000004 | Cout: 0
    Actual   -> Sout: 0x00000004 | Cout: 0

// Test 15: j=3 (A0=0x00000007, CIN=1)
[PASS] @ 149.9 ns
    Expected -> Sout: 0x00000008 | Cout: 0
    Actual   -> Sout: 0x00000008 | Cout: 0

// Test 16: j=4 (A0=0x0000000F, CIN=1)
[PASS] @ 159.9 ns
    Expected -> Sout: 0x00000010 | Cout: 0
    Actual   -> Sout: 0x00000010 | Cout: 0

// Test 17: j=5 (A0=0x0000001F, CIN=1)
[PASS] @ 169.9 ns
    Expected -> Sout: 0x00000020 | Cout: 0
```

Actual -> Sout: 0x00000020 | Cout: 0

// Test 18: j=6 (A0=0x0000003F, CIN=1)

[PASS] @ 179.9 ns

Expected -> Sout: 0x00000040 | Cout: 0

Actual -> Sout: 0x00000040 | Cout: 0

// Test 19: j=7 (A0=0x0000007F, CIN=1)

[PASS] @ 189.9 ns

Expected -> Sout: 0x00000080 | Cout: 0

Actual -> Sout: 0x00000080 | Cout: 0

// Test 20: j=8 (A0=0x000000FF, CIN=1)

[PASS] @ 199.9 ns

Expected -> Sout: 0x00000100 | Cout: 0

Actual -> Sout: 0x00000100 | Cout: 0

// Test 21: j=9 (A0=0x000001FF, CIN=1)

[PASS] @ 209.9 ns

Expected -> Sout: 0x00000200 | Cout: 0

Actual -> Sout: 0x00000200 | Cout: 0

// Test 22: j=10 (A0=0x000003FF, CIN=1)

[PASS] @ 219.9 ns

Expected -> Sout: 0x00000400 | Cout: 0

Actual -> Sout: 0x00000400 | Cout: 0

// Test 23: j=11 (A0=0x000007FF, CIN=1)

[PASS] @ 229.9 ns

Expected -> Sout: 0x00000800 | Cout: 0

Actual -> Sout: 0x00000800 | Cout: 0

// Test 24: j=12 (A0=0x00000FFF, CIN=1)

[PASS] @ 239.9 ns

Expected -> Sout: 0x00001000 | Cout: 0

Actual -> Sout: 0x00001000 | Cout: 0

// Test 25: j=13 (A0=0x00001FFF, CIN=1)

[PASS] @ 249.9 ns

Expected -> Sout: 0x00002000 | Cout: 0

Actual -> Sout: 0x00002000 | Cout: 0

// Test 26: j=14 (A0=0x00003FFF, CIN=1)

[PASS] @ 259.9 ns

Expected -> Sout: 0x00004000 | Cout: 0
Actual -> Sout: 0x00004000 | Cout: 0

// Test 27: j=15 (A0=0x00007FFF, CIN=1)
[PASS] @ 269.9 ns
Expected -> Sout: 0x00008000 | Cout: 0
Actual -> Sout: 0x00008000 | Cout: 0

// Test 28: j=16 (A0=0x0000FFFF, CIN=1)
[PASS] @ 279.9 ns
Expected -> Sout: 0x00010000 | Cout: 0
Actual -> Sout: 0x00010000 | Cout: 0

// Test 29: j=17 (A0=0x0001FFFF, CIN=1)
[PASS] @ 289.9 ns
Expected -> Sout: 0x00020000 | Cout: 0
Actual -> Sout: 0x00020000 | Cout: 0

// Test 30: j=18 (A0=0x0003FFFF, CIN=1)
[PASS] @ 299.9 ns
Expected -> Sout: 0x00040000 | Cout: 0
Actual -> Sout: 0x00040000 | Cout: 0

// Test 31: j=19 (A0=0x0007FFFF, CIN=1)
[PASS] @ 309.9 ns
Expected -> Sout: 0x00080000 | Cout: 0
Actual -> Sout: 0x00080000 | Cout: 0

// Test 32: j=20 (A0=0x000FFFFFF, CIN=1)
[PASS] @ 319.9 ns
Expected -> Sout: 0x00100000 | Cout: 0
Actual -> Sout: 0x00100000 | Cout: 0

// Test 33: j=21 (A0=0x001FFFFFF, CIN=1)
[PASS] @ 329.9 ns
Expected -> Sout: 0x00200000 | Cout: 0
Actual -> Sout: 0x00200000 | Cout: 0

// Test 34: j=22 (A0=0x003FFFFFF, CIN=1)
[PASS] @ 339.9 ns
Expected -> Sout: 0x00400000 | Cout: 0
Actual -> Sout: 0x00400000 | Cout: 0

// Test 35: j=23 (A0=0x007FFFFFF, CIN=1)

[PASS] @ 349.9 ns

Expected -> Sout: 0x00800000 | Cout: 0

Actual -> Sout: 0x00800000 | Cout: 0

// Test 36: j=24 (A0=0x00FFFFFF, CIN=1)

[PASS] @ 359.9 ns

Expected -> Sout: 0x01000000 | Cout: 0

Actual -> Sout: 0x01000000 | Cout: 0

// Test 37: j=25 (A0=0x01FFFFFF, CIN=1)

[PASS] @ 369.9 ns

Expected -> Sout: 0x02000000 | Cout: 0

Actual -> Sout: 0x02000000 | Cout: 0

// Test 38: j=26 (A0=0x03FFFFFF, CIN=1)

[PASS] @ 379.9 ns

Expected -> Sout: 0x04000000 | Cout: 0

Actual -> Sout: 0x04000000 | Cout: 0

// Test 39: j=27 (A0=0x07FFFFFF, CIN=1)

[PASS] @ 389.9 ns

Expected -> Sout: 0x08000000 | Cout: 0

Actual -> Sout: 0x08000000 | Cout: 0

// Test 40: j=28 (A0=0x0FFFFFFF, CIN=1)

[PASS] @ 399.9 ns

Expected -> Sout: 0x10000000 | Cout: 0

Actual -> Sout: 0x10000000 | Cout: 0

// Test 41: j=29 (A0=0x1FFFFFFF, CIN=1)

[PASS] @ 409.9 ns

Expected -> Sout: 0x20000000 | Cout: 0

Actual -> Sout: 0x20000000 | Cout: 0

// Test 42: j=30 (A0=0x3FFFFFFF, CIN=1)

[PASS] @ 419.9 ns

Expected -> Sout: 0x40000000 | Cout: 0

Actual -> Sout: 0x40000000 | Cout: 0

// Test 43: j=31 (A0=0x7FFFFFFF, CIN=1)

[PASS] @ 429.9 ns

Expected -> Sout: 0x80000000 | Cout: 0

Actual -> Sout: 0x80000000 | Cout: 0

SUMMARY: 43 out of 43 tests passed at VDD = 0.32 V
>>> RESULT: VDD = 0.32 V is a VALID operating point <<<

=====
Verification Complete.
=====

Leakage power

=====
Calculating Leakage Power at VDD = 0.32 V
=====

--- Static Leakage Power Results ---

State 1 (All Zeros) : 3.083720e-07 W

State 2 (All Ones) : 5.338124e-07 W

State 3 (Propagate 0): 8.058070e-07 W

State 4 (Propagate 1): 8.064468e-07 W

>>> MINIMUM LEAKAGE POWER: 3.083720e-07 W
>>> MAXIMUM LEAKAGE POWER: 8.064468e-07 W

=====
Leakage Calculation Complete.
=====

Average power, PDP, EDP

=====
Calculating Average Metrics at VDD = 0.32 V
=====

>>> AVERAGE TOTAL POWER: 2.902443e-06 W
>>> AVERAGE PDP: 2.902443e-14 J
>>> AVERAGE EDP: 2.902443e-22 J*s

Average Calculation Complete.

Max frequency at each voltage level (0.1-1v at 0.1v increments)

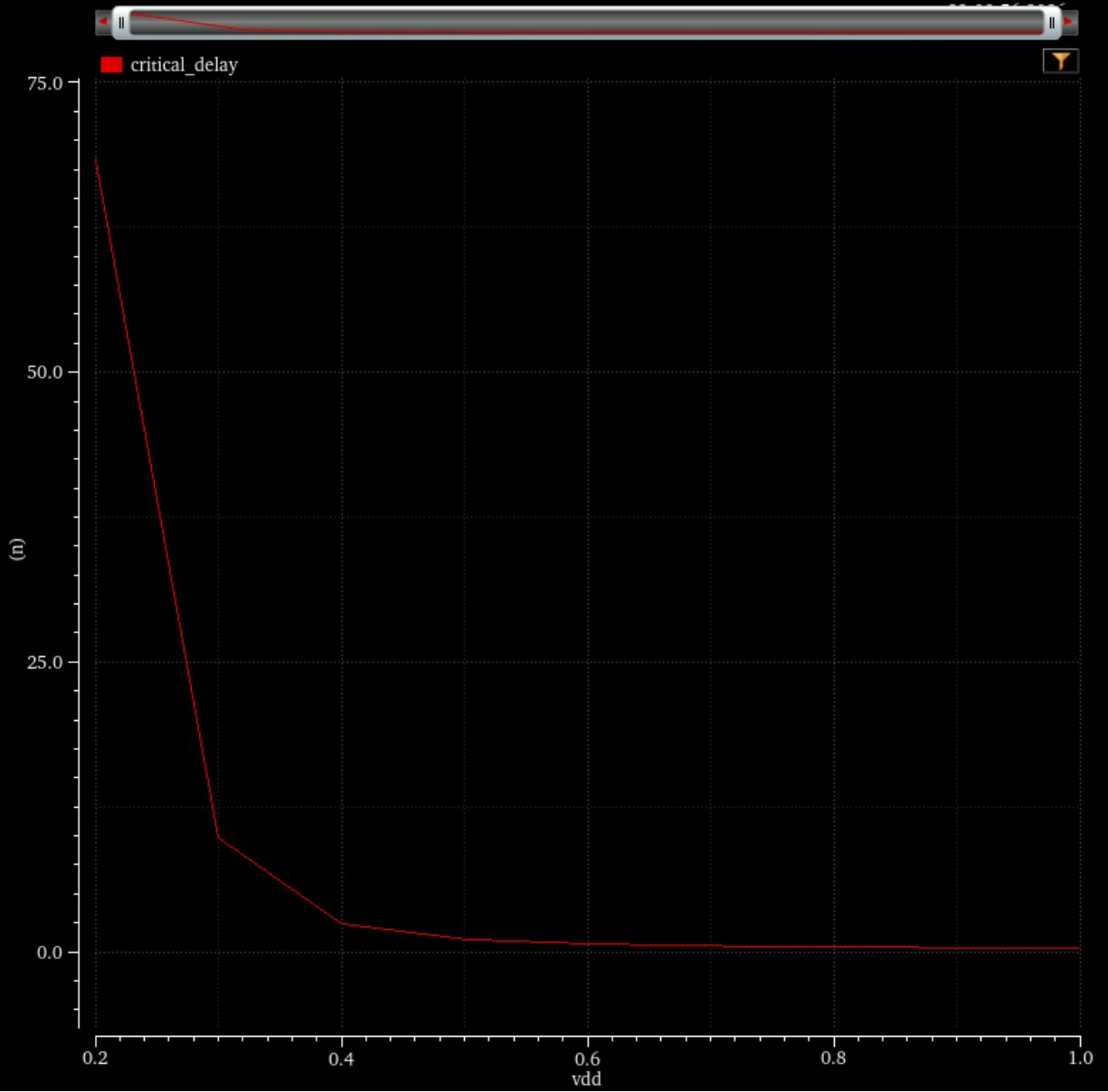
delay(?wf1 VT("/CINin"), ?value1 VAR("vdd")/2, ?edge1 "rising", ?nth1 1, ?td1 0.0, ?tol1 nil, ?wf2 VT("/Sout<31>"), ?value2 VAR("vdd")/2, ?edge2 "falling", ?nth2 1, ?tol2 nil, ?td2 nil, ?stop nil, ?multiple nil)

VDD (V)	Critical Delay (Raw s)	Critical Delay (ns)	Fmax (MHz)	Notes
0.1	eval err	-	-	Fails (Below Vmin)
0.2	6.85E-08	68.54	14.59	-
0.3	9.84E-09	9.842	101.61	-
0.4	2.42E-09	2.421	413.05	-
0.5	1.08E-09	1.08	925.93	-
0.6	6.82E-10	0.6815	1467.35	-
0.7	5.06E-10	0.5059	1976.68	-
0.8	4.12E-10	0.4121	2426.6	-
0.9	3.58E-10	0.3575	2797.2	-
1	3.24E-10	0.3238	3088.33	Nominal Voltage

critical_delay

Tue Mar 10

1



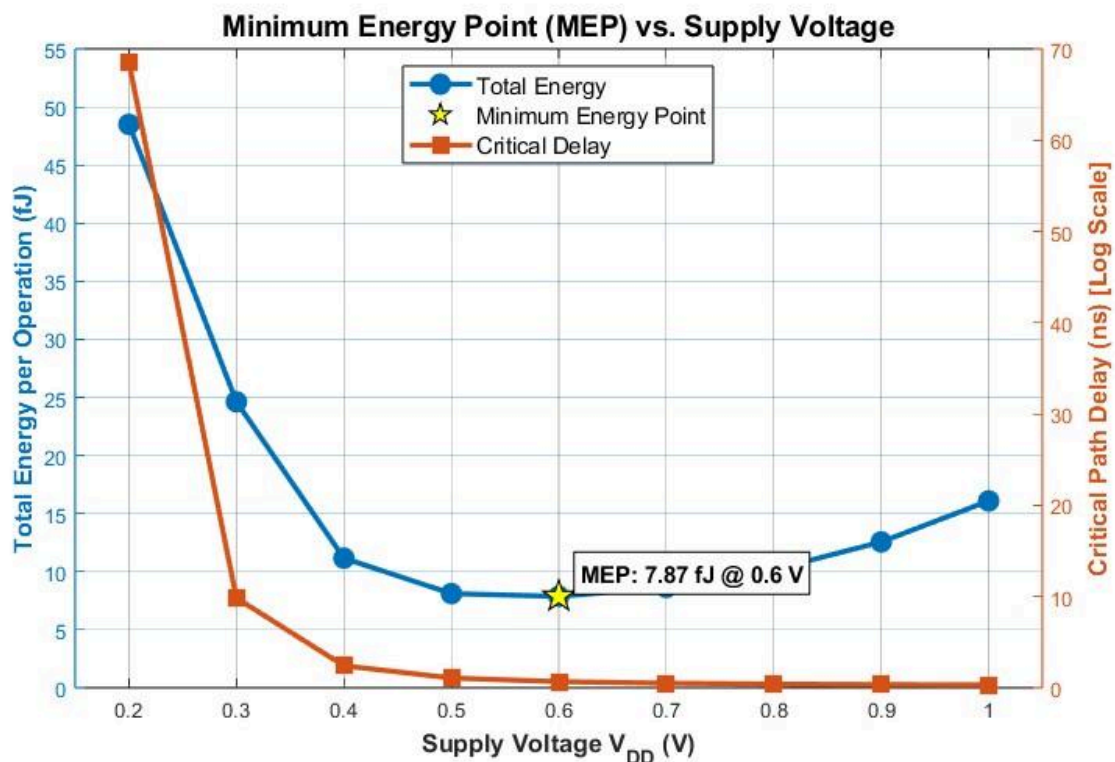
Energy given frequency

calc_mep_sweep

Auto-detected latest sweep run: ExplorerRun.0

VDD(V) | Avg Power (W) | Critical Delay (s) | Energy (J)

0.1	8.3434e-08	FAILED	FAILED
0.2	7.0774e-07	6.8540e-08	4.8508e-14
0.3	2.5023e-06	9.8420e-09	2.4628e-14
0.4	4.6114e-06	2.4210e-09	1.1164e-14
0.5	7.5131e-06	1.0800e-09	8.1141e-15
0.6	1.1553e-05	6.8150e-10	7.8732e-15
0.7	1.7092e-05	5.0590e-10	8.6471e-15
0.8	2.4685e-05	4.1210e-10	1.0173e-14
0.9	3.5137e-05	3.5750e-10	1.2562e-14
1.0	4.9704e-05	3.2380e-10	1.6094e-14



References

1. C. Kamble, R. K. Siddharth, S. Patidar, M. H. Vasantha and K. Y. B. Nithin, "Design of Area-Power-Delay Efficient Square Root Carry Select Adder," 2018 IEEE International Symposium on Smart Electronic Systems (iSES) (Formerly iNiS), Hyderabad, India, 2018, pp. 80-85, doi: 10.1109/iSES.2018.00026. keywords: {Adders;Delays;Computer

architecture;Complexity theory;Transistors;Multiplexing;Logic gates;Adders;power-delay product (PDP);delay;low power},

2. “Design of 32 Bit Low Power and High Speed Square Root Carry Select Adder,” *Ijrras.com*, 2022.
<http://ijrras.com/design-of-32-bit-low-power-and-high-speed-square-root-carry-select-adder/> (accessed Mar. 11, 2026).
3. Y. Ykuntam, M. Rao, and G. Locharla, “Design of 32-bit Carry Select Adder with Reduced Area,” *International Journal of Computer Applications*, vol. 75, no. 2, pp. 975–8887, 2013, Accessed: Mar. 11, 2026. [Online]. Available:
<https://research.ijcaonline.org/volume75/number2/pxc3890353.pdf>